

Time 1:30 h. -- --

1. (6 points) We need that the first half of the layers be of 32 heads and the other half be of only 2 heads. Write in detail where and how the program on the book has to be modified to do so. Say if the number of learned parameters during training would be equal, bigger or smaller than in the original implementation, justifying the answer.

Sol:

The lines with the changes are marked with comments. There is no change in the number of learned parameters because the number of parameters for the matrix q , for instance, is $(768 \times 768 / \text{numheads})$ and there are numheads matrices so, there are 768×768 parameters.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, config, firsthalf): ### for exam
        super().__init__()
        embed_dim = config.hidden_size
        #num heads = config.num attention heads
        if (firsthalf): ##### for exam
            num_heads=32
        else:
            num_heads=2
        head_dim = embed_dim // num_heads
        self.heads = nn.ModuleList(
            [AttentionHead(embed_dim, head_dim) for _ in range(num_heads)]
        )
        self.output_linear = nn.Linear(embed_dim, embed_dim)

    def forward(self, hidden_state):
        x = torch.cat([h(hidden_state) for h in self.heads], dim=-1)
        x = self.output_linear(x)
        return x

class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.linear_1 = nn.Linear(config.hidden_size, config.intermediate_size)
        self.linear_2 = nn.Linear(config.intermediate_size, config.hidden_size)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(config.hidden_dropout_prob)

    def forward(self, x):
        x = self.linear_1(x)
        x = self.gelu(x)
        x = self.linear_2(x)
        x = self.dropout(x)
        return x

class TransformerEncoderLayer(nn.Module):
    def __init__(self, config, firsthalf): ##### for exam
        super().__init__()
        self.layer_norm_1 = nn.LayerNorm(config.hidden_size)
```

```

        self.layer_norm_2 = nn.LayerNorm(config.hidden_size)
        self.attention = MultiHeadAttention(config, first_half) ##### for exam
        self.feed_forward = FeedForward(config)

    def forward(self, x):
        # Apply layer normalization and then copy input into query, key, value
        hidden_state = self.layer_norm_1(x)
        # Apply attention with a skip connection
        x = x + self.attention(hidden_state)
        # Apply feed-forward layer with a skip connection
        x = x + self.feed_forward(self.layer_norm_2(x))
        return x

class Embeddings(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.token_embeddings = nn.Embedding(config.vocab_size,
                                              config.hidden_size)
        self.position_embeddings = nn.Embedding(config.max_position_embeddings,
                                              config.hidden_size)
        self.layer_norm = nn.LayerNorm(config.hidden_size, eps=1e-12)
        self.dropout = nn.Dropout()

    def forward(self, input_ids):
        # Create position IDs for input sequence
        seq_length = input_ids.size(1)
        position_ids = torch.arange(seq_length, dtype=torch.long).unsqueeze(0)
        # Create token and position embeddings
        token_embeddings = self.token_embeddings(input_ids)
        position_embeddings = self.position_embeddings(position_ids)
        # Combine token and position embeddings
        embeddings = token_embeddings + position_embeddings
        embeddings = self.layer_norm(embeddings)
        embeddings = self.dropout(embeddings)
        return embeddings

class TransformerEncoder(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embeddings = Embeddings(config)
        half_layers = config.num_hidden_layers // 2 ##### for exam
        self.layers = nn.ModuleList([TransformerEncoderLayer(config, (i < half_layers))
                                      for i in range(config.num_hidden_layers)])

    def forward(self, x):
        x = self.embeddings(x)
        for layer in self.layers:
            x = layer(x)
        return x

```

2. (4 points) The output token of the decoder is the one with the highest probability. Write in detail where and how the program on the Full Implementation of the Transformer has to be modified so that the output token is a sample that follows the distribution probabilities but not necessarily the one with the highest probability.

Sol:

Replace the line 1001 by

```
prob = prob.softmax()  
next word = multinomial(prob)
```

The function softmax converts the output numbers to probabilities. The function multinomial(p) takes a vector of probabilities and returns an index on it following the probabilities (the implementation could be to generate a random number r between 0 and 1, and then start adding the probabilities from the beginning of the vector and when the sum arrives to r , return the vector index where that happens).